Reviewer Pack — Minimal Lattice Point Counting and Communication Complexity …

Entry 9956de739341 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 00:50:23 UTC

Minimal Lattice Point Counting and Communication Complexity Rank Inequality

Entry ID: `9956de739341` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 00:50:23 UTC

1. Conjecture

Field A (mathematical branch): Discrete Geometry (Lattice Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every boolean function f with n inputs, the number of lattice points inside or on the convex hull of the image of f in R^n is at most a constant times the rank of the corresponding matrix representation of f.

Rationale (proposer's reasoning):

> This conjecture bridges lattice theory, which studies discrete structures, with communication complexity, which studies distributed computation. It suggests that the geometric structure of boolean functions could be related to their computational complexities in distributed settings.

Taxonomy category: `LATTICE_TO_COMMUNICATION_RANK` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `de2b67d0113db327`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For boolean function f with n inputs, if the ratio of the average lattice point count inside or on the convex hull of f's image to the average rank of its matrix representation is ≤ k for some constant k and all seeds, then the conjecture is supported. If any seed results in this ratio > k or any seed produces a single function with more lattice points than 1.5 times the corresponding rank, the conjecture is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"lattice point counting" AND "communication complexity" AND "matrix rank"` - `"convex hull" IN DISCRIMINATEGeometry AND communication complexity AND rank inequality"` - `"Discrete Geometry" AND lattice theory AND matrix representation communication complexity`

Top relevant hits considered: - [s2:10.1007/978-3-642-12200-2] LATIN 2010: Theoretical Informatics, 9th Latin American Symposium, Oaxaca, Mexico, April 19-23, 2010. Proceedings - [s2:12bf5fa50a22abca567ba21cf0ad5ba4908a9826] Computing and combinatorics : 14th annual international conference, COCOON 2008, Dalian, China, June 27-29, 2008 : proce - [s2:edb976a1b7c29e5ab325ed016a26e03f0b324a5f] A Short History of Symplectic Dynamic

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=-9, elapsed=130.9s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def convex_hull_lattice_points(f):
        n = len(f)
        points = [(i & (1 << j)) for i in range(2**n) for j in range(n)]
        hull = set()
        for point in points:
            if all(point[i] <= f[point[i]] for i in range(n)):
                hull.add(tuple(sorted(point)))
        return len(hull)

    def matrix_representation(f):
        n = len(f)
        A = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                if all((i & (1 << k)) <= f[(j & (1 << k))] for k in range(n)):
                    A[i][j] = 1
        return A
```

```
def rank_of_matrix(A):
    n = len(A)
    m = len(A[0])
    M = [A[i] + [-i] for i in range(n)]
    r = 0
    for j in range(m):
        if all(M[i][j] == 0 for i in range(r, n)):
            continue
        pivot_row = r
        while M[pivot_row][j] == 0:
            pivot_row += 1
            if pivot_row == n:
                break
        M[pivot_row], M[r] = M[r], M[pivot_row]
        for i in range(r + 1, n):
            factor = -M[i][j] / M[r][j]
            for k in range(m):
                M[i][k] += factor * M[r][k]
        r += 1
    return r

def generate_random_integers(k):
    return [random.randint(0, 2**31 - 1) for _ in range(k)]

n_values = [5, 10, 15, 20, 30, 40]
total_points = 0
total_rank = 0
instances_tested = 0

for n in n_values:
    for _ in range(5):  # Ensure at least 30 instances per seed
        f = generate_boolean_function(n)
        points = convex_hull_lattice_points(f)
        A = matrix_representation(f)
        rank = rank_of_matrix(A)
        total_points += points
        total_rank += rank
        instances_tested += 1

if instances_tested < 30:
    return {
        "metric_name": "lattice_point_count_to_rank_ratio",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

ratio = total_points / total_rank
if ratio > 1.5:
    return {
        "metric_name": "lattice_point_count_to_rank_ratio",
        "metric_value": ratio,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
```

```
            "conjecture_holds": False,
            "counterexample": f"ratio={ratio} exceeds 1.5"
        }

    return {
        "metric_name": "lattice_point_count_to_rank_ratio",
        "metric_value": ratio,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **result}}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_ratio = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] and "counterexample" in r for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = next(result["counterexample"] for result in results if "counterexample" i
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
(empty)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means we cannot verify the conjecture with the provided pre-registered support conditions. | next: Re-run the test to ensure it completes successfully and provides results.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17205
2	propose	ollama_remote	glm4:latest	0	0	11854
3	propose	ollama_remote	glm4:latest	0	0	12053
4	preregistration	ollama_remote	glm4:latest	0	0	10117
5	novelty	ollama_remote	glm4:latest	0	0	8420
6	novelty	ollama_remote	glm4:latest	0	0	9370
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43861
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8789
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12944
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12870
11	judge	ollama_remote	glm4:latest	0	0	80433

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 227916 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/9956de739341.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9956de739341.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9956de739341.tar.gz` (if generated)